

Gym Slot Booking

Take-home challenge. Any stack you can explain.

The problem

A gym runs one-hour sessions, each holding 20 people. Bookings are taken by phone and written in a notebook, which causes overbooking and arguments at the door.

Build the gym's booking system: members book and cancel sessions online, and staff mark who turned up.

What to deliver

- A working backend API with a real database
- A working frontend web application
- A Git repository with real commit history and a README
- A design note in the README, ten lines at most, on your stack and data model
- Optional: deploy it. Free tiers are fine - Vercel or Netlify for the frontend, Render or Railway for the API, Supabase or Neon for the database.

Rules

- Any language, framework and database. We evaluate your reasoning, not the brand.
- You must be able to explain and modify every line you submit. A live change request is part of the review.
- Commit as you work. A single commit at the end is marked down.
- No secrets in the repository. Use environment variables and include `.env.example`.
- Where something is ambiguous, decide, note it in the README under Assumptions, and carry on. Questions cannot be answered during the challenge.

What to build

Accounts

- Register with name, email and password. Email is unique.
- Log in and receive a token. Passwords are hashed, never stored as text.
- Two roles: members book for themselves, admins run the gym. Admin accounts are created directly in the database, never through registration.

Sessions

- An admin creates sessions with a date, a start hour and a capacity of 20.
- Sessions start on the hour and last one hour. Two cannot share a start time.
- Anyone logged in can browse sessions with seats remaining, and filter by date.

Bookings

- A member books a session that has not started and has a free seat.
- A session never exceeds 20 confirmed bookings, **even when two people take the last seat at the same moment**.
- One active booking per member per session.
- A member can cancel until the session starts, and can rebook later if seats remain.
- Members see only their own bookings. Admins see all bookings for a session.

Waitlist

- When a session is full, a member can join its waitlist instead.
- When someone cancels, the first person on the waitlist is booked automatically and their place on the list is released.
- A member can leave the waitlist, and cannot be on it twice for the same session.
- Nobody is promoted into a session that has already started.

Check-in

- Each confirmed booking gets a short code from the server, for example 7XKP2Q. The member sees it in the app.
- An admin types the code in to mark that member present, and sees whose booking it is.
- A code works once, only during its session's hour, and never for a cancelled booking.
- Codes must not be guessable from each other.

How it must behave

API

- Correct status codes: 400 or 422 for invalid input, 401 unauthenticated, 403 forbidden, 404 not found, 409 for conflicts such as a full session.
- Every route validates its input on the server. One consistent JSON error format.
- A global error handler. The API never crashes or leaks stack traces.
- Auth middleware on every non-public route, role checks on admin routes, and ownership checks so members reach only their own data. Hiding a button is not authorization.

Data

- A real database with related tables, not JSON files or in-memory arrays.

Frontend

- Four pages: login and register, sessions list, my bookings, and one admin page.
- The admin page does three things: create a session, see a session's bookings, and mark who turned up. It can look plain; we are not marking the styling.
- Client-side routing with protected routes. A member cannot open the admin page.
- Every screen that loads data shows loading, error, empty and success states. Buttons disable while a request is in flight.
- Forms validate inline, and the server validates independently.
- Booking feels instant: the seat count updates straight away and rolls back if the server refuses.
- Seats remaining update while the page is open, without a refresh. Polling is fine.
- Usable at 360px, operable by keyboard, every input labelled, errors written in text and not signalled by colour alone.

Test data

Leave the app with something in it: an admin account, a couple of members, and sessions across a few days including one that is full. Put the login details in your README.

What to submit

Repository URL, and admin and member test credentials in the README. If you deployed it, send the live links too.